

EXACT Pipeline: Solution Description

1. Datasets Used

1.1. Official EXACT Dataset

- **Name:** EXACT Official Benchmark Dataset.
- **Source/Origin:** Official EXACT competition dataset.

1.2. External Knowledge Corpus (Graph & Vector DB)

- **Name:** Neuro-Symbolic Physics & Logic Rules Corpus.
 - **Source/Origin:** Synthesized logic rules and physics formulas extracted from open-source scientific libraries and reasoning datasets (based-on EXACT Dataset), structured to populate our Vector Database (**ChromaDB**) and Knowledge Graph (**NetworkX**).
-

2. Approach and Method

The workflow is divided into five main phases:

Phase 0: Offline Ingestion & Knowledge Compilation

Before query processing, natural language rules, math formulas, and logic premises are pre-compiled. Formulas and rules are structured into a Neuro-Symbolic **HybridDB** consisting of:

- **VectorDB (**ChromaDB**):** Embeds textual definitions via **BAAI/bge-small-en-v1.5** to allow fast semantic retrieval.
- **GraphDB (**NetworkX**):** Connects concepts topologically, using PageRank to prioritize relevant subgraphs and causal relationships during reasoning.

Phase 1: Hardware-Aware Adaptive Routing & Semantic Extraction

When a query is received, the system evaluates its metadata and context without invoking any heavy LLM layers:

- **Static NLP & Complexity:** Calculates query complexity based on token counts, structure indices, and syntactic hops (e.g., keywords like **and**, **then**, **if**).
- **Utility Maximization:** An Exponential Moving Average (EMA) utility model computes whether to execute the query via the **Fast Path** (direct formula/cache lookups) or the **Hybrid Path** (involving LLM query expansion and code generation).

Phase 2: Hybrid Retrieval & RAG-Based FOL Cache

For queries routed to the hybrid path, the retrieval layer gathers high-quality context:

- **Semantic Expansion:** Uses the Main LLM to extract named entities and generate search anchors (HyDE contexts).

- **Dense & Subgraph Retrieval:** Fetches the top vector matches from **ChromaDB** and extracts neighboring nodes in the **GraphDB** using Breadth-First Search (BFS) and Steiner Tree approximations.
- **RAG-Based FOL Cache:** If the natural language premises in the query have been translated previously, the system maps and retrieves the pre-translated FOL rules from the HybridDB, avoiding redundant LLM calls and ensuring maximum syntax consistency.

Phase 3: Domain-Specific Reasoning Pipelines

Type 1: Logic Pipeline

Based on the query type (e.g., yes/no/uncertain, multiple-choice, open-ended), the reasoning pipeline runs as follows:

- **Horn Parser (Fast Path):** If the premises are mapped to definite Horn clauses (simple chaining rules - backward & forward chaining), the pipeline executes a Horn Parser - Deterministic Reasoning engine to prove the target, bypassing Z3 and the LLM completely.
- **Full Symbolic Path (AST + Z3):** If the query is complex or non-Horn:
 1. The Main LLM (**Qwen3-8B**) translates the natural language premises and target query into a structured, validated JSON First-Order Logic (FOL) Abstract Syntax Tree (AST) or prefix token stream.
 2. A **FOL Post-Processing Sanitizer** applies pattern-based fixes to resolve common LLM translation errors (e.g., mapping tautologies like $P(x) \rightarrow P(x)$ to standalone facts, fixing double-predicate applications, and wrapping unbound free variables with universal quantifiers).
 3. The parsed AST is compiled into Z3 SMT constraints.
 4. The **Symbolic Z3 Solver** evaluates the assertions using Proof by Contradiction and SAT solving.
- **LLM Text Fallback:** If Z3 verification fails or the schema is unresolvable, the system falls back to text-based reasoning via Qwen3.

Type 2: Physics Pipeline

Calculates numeric values or extracts formulas using the following mechanisms:

- **Fast Cache Bypass:** Masks numbers within the query - **SymPy Solver Fast Path:** Under the adaptive router's fast-path instruction, the system retrieves matching formulas from the HybridDB and substitutes input values directly using SymPy, solving the query algorithmically.
- **LLM Code Generation & Self-Repair Loop:** If the problem is complex, a Jinja2 template prompts the Main LLM to write a self-contained Python script utilizing SymPy.

Phase 4: Deterministic Sandbox Execution & Feedback Loops

To guarantee system security and deterministic correctness:

- **AST Safety Inspection:** All generated Python and Z3 scripts are parsed into Python ASTs and inspected before execution. Dangerous packages/methods (`import os, sys, exec, eval`) are blocked.
- **Isolated Execution:** Scripts run in an isolated child process using a **PythonSandboxExecutor** or **Z3Executor** with a strict execution timeout (4.0s).

3. Model Size Calculation

To comply with the **8B parameter limit**, our pipeline utilizes a **Single-LLM architecture** where all generative tasks (orchestration, FOL AST translation, code generation) are centralized within a single active model.

Generative Models:

1. **Main Generator LLM:** `Qwen3-8B`

- **Parameter Count:** **~8.00 B.**
- **Role:** Handles orchestration, semantic extraction, logic AST translation, and Python code generation.

Auxiliary Models (Non-Generative Encoders):

1. **Dense Retriever:** `BAAI/bge-small-en-v1.5`

- **Parameter Count:** **~0.03 B**
- **Role:** Computes text embeddings for ChromaDB vector search. Bypassed via a fast lexical index if the model is not loaded locally.

2. **Semantic Reranker:** `BAAI/bge-reranker-v2-m3`

- **Parameter Count:** **~0.56 B**
- **Role:** Performs cross-encoder scoring of retrieved contexts. Falls back to raw ChromaDB cosine rankings if sentence-transformers is offline.

Active Parameter Count Calculation:

- Since all generative tasks are routed to the **single model** `Qwen3-8B`, the running parameter count for generation at any given moment is exactly **~8.00 B**.
- The auxiliary encoder models (`bge-small` and `bge-reranker`) are non-generative, run sequentially, and can be completely disabled/bypassed using local lexical fallbacks.
- Consequently, the maximum running parameter size under server load is strictly capped at the **8B-class limit**, in full compliance with the official competition requirements. No Mixture of Experts (MoE) or multi-LLM architectures are deployed.